

Projet DevOps

Vous allez durant ce projet réviser l'ensemble des concepts que nous avons vu dans le cours. Vous travaillerez à deux sur le projet et envoyer un dossier zipper qui contient toutes les configurations.

La configuration que vous allez fournir devrai être exécutable dans n'importe quel environnement Docker et d'autre part dans n'importe quel cluster Kubernetes sans problème

Architecture :

Cette application web permet aux utilisateurs d'entrer leur prénom, nom et âge pour obtenir une prédiction humoristique de leur date de décès. L'application permet également de lister toutes les prédictions faites, de les supprimer, et de naviguer entre différentes pages.

Structure de l'Application

L'application est divisée en trois parties principales :

1. **Backend (Flask)** : Gère les requêtes API, interagit avec la base de données MySQL et génère les prédictions.
2. **Frontend (React)** : Fournit une interface utilisateur permettant d'entrer des données, de voir les résultats de prédictions et de gérer la liste des prédictions.
3. **Docker** : Utilisé pour conteneuriser l'application, ce qui facilite le déploiement et la gestion des dépendances.

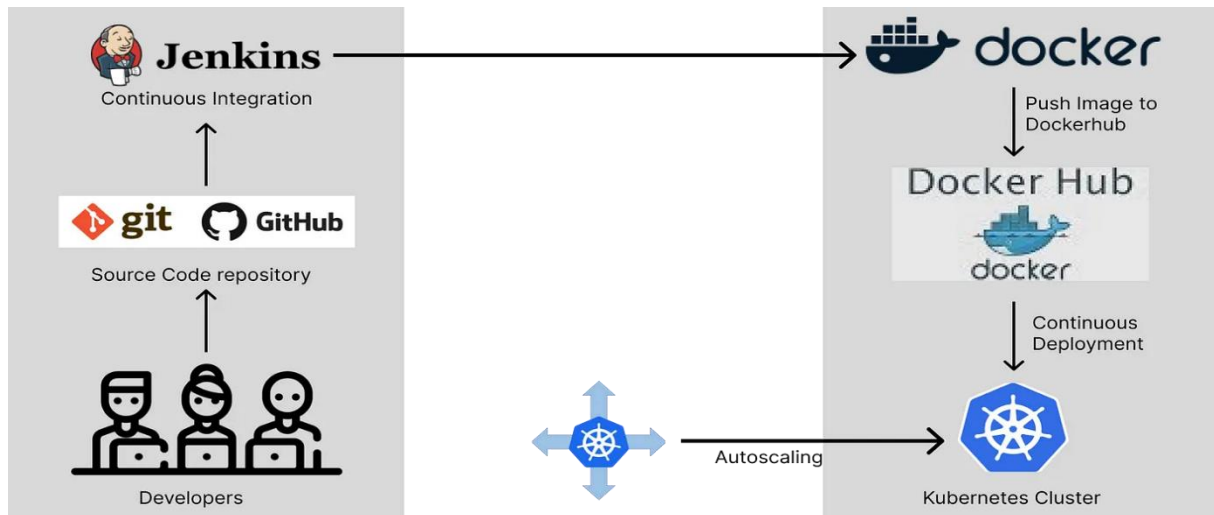
Structure des Répertoires

```
ml_project/
├── backend/
│   ├── app/
│   │   ├── __init__.py
│   │   ├── models.py
│   │   ├── routes.py
│   │   └── ml_model.py
│   ├── Dockerfile
│   ├── requirements.txt
│   └── run.py
├── frontend/
│   ├── public/
│   │   └── index.html
│   ├── src/
│   │   ├── components/
│   │   │   ├── HomePage.js
│   │   │   ├── ResultPage.js
│   │   │   └── PredictionList.js
│   │   ├── App.js
│   │   ├── App.css
│   │   └── index.js
│   └── Dockerfile
├── docker-compose.yml
└── .env
```

Le projet est déjà structuré avec leurs Dockerfiles respectives. J'ai aussi fourni un **docker-compose** pour le lancer et vous pouvez tester en vous positionnant dans le répertoire **ml_project** et exécuter **docker-compose up - --build** ou (**docker compose up - --build**)

Tâches à réaliser :

Vous allez déployer cette application en utilisant une chaîne CI/CD c'est-à-dire utiliser git-jenkins-docker-kubernetes comme le montre le schéma suivant



Vous avez déjà les Dockerfiles pour le backend et le frontend. Vous aurez besoin de builder et d'envoyer les images vers votre registry dockerhub pour pouvoir les utiliser dans vos déploiements. Créez un dossier manifeste dans chaque partie où vous allez mettre les configs de déploiements (backend et frontend), référez-vous au cours.

1. Créer un namespace **ml_project** avec un fichier de config
2. Créer trois fichiers de déploiement :
 - a. Un **déploiement** pour la base de données MySQL
 - b. Un **déploiement** pour l'application backend
 - c. Un **déploiement** pour l'application frontend
3. Créer trois services :
 - a. Un service de type **ClusterIP** pour la base de données MySQL (Ce service est utilisé par le backend pour se connecter à la base de données)
 - b. Un service de type **ClusterIP** pour l'application backend qui expose le port 5000 (Ce service est utilisé par l'application frontend pour appeler les API)
 - c. Un service de type **NodePort** qui target le port 3000 et expose le port 30001 et qui servira à accéder sur l'application depuis le navigateur
4. Créer trois configMaps :
 - a. Un **configMap** pour la base de données MySQL qui stock (le nom de la base de données **MYSQL_DATABASE**)
 - b. Un **configMap** pour l'application backend qui stock (le nom de la base de données **MYSQL_DATABASE** et la variable d'environnement **FLASK_ENV** que vous trouverez dans le docker-compose)

- c. Un **configMap** pour l'application frontend qui stock (l'url du backend, ici vous devez veillez à utiliser le nom du service du backend avec le port 5000 exposé et la variable d'environnement CHOKIDAR_USEPOLLING)
- 5. Créer un secret :
 - a. Le **secret** que vous allez créer doit être partagé en même temps par la base de données MYSQL et l'application backend, référez-vous au docker-compse.yml pour observer ces variables d'env (MYSQL_ROOT_PASSWORD, MYSQL_USER, MYSQL_PASSWORD)
- 6. Créer un **PVC** de 2Gi pour stocker les données de la base de données (Pensez à monter ce PVC dans le déploiement du MySQL)

Une fois que vous aurez fini de créer tous les fichiers de configurations, je devrais être en mesure de les lancer sans problème.